

レポート課題 5

05-191009 奥村真司

問 1

動作例

```
# fun x -> x+y;;  
- = <fun>  
# (fun x -> x+5) 3;;  
- = 8  
# (let y=5 in fun x -> x+y) 3;;  
- = 8
```

考察

講義スライドに従って関数のクロージャを作成し、引数を呼び出し時の環境で評価し、それで拡張した呼び出し時の環境のもとで式の部分を評価した。

問 2

動作例

```
# let rec f n = if n = 1 then 1 else n + f (n-1);;  
val f = <rec fun>  
# f 10;;  
- = 55  
# f 15;;  
- = 120
```

考察

講義スライドの 1 つ目で扱った評価時に工夫する方針で実装した。クロージャを通常関数とは別の `VRecFun` という名前で作くり (構成は `VFun` と同じ)、式部分の評価時に引数だけでなく自分自身でも拡張した環境を用いるようにした。

問 3

動作例

```
# let rec odd n = if n = 1 then true else if n = 0 then false else even (n-1)
and even n = if n = 0 then true else if n = 1 then false else odd (n-1);;
rec functions = <fun>
# odd 15;;
- = true
# odd 20;;
- = false
# even 131;;
- = false
# even 2000;;
- = true
```

考察

問 3 の講義スライドの方法 1 で実装した。方針としては問 2 で 1 つの再帰関数を実装したときのものを拡張しただけで、 n 個の相互再帰する関数をまとめて扱って、評価時に自分自身を全て環境に追加してやっているだけである。Parser を書き換えた結果、問 2 での 1 つの再帰関数定義 `let rec...` は `let rec ... and ... and ...` の `and` が一つも続かないものの一つとして相互再帰の構文の一つとして扱われている。

問 4

動作例

```
# let rec f n = if n = 0 then 0 else n + f (n-1);;
rec val f = <fun>
# f 10;;
- = 55
# f 20;;
- = 210
```

考察

講義スライドの方法 2 で実装した。追加した関数のクロージャの環境がその関数が定義され拡張されたあとの環境自身を指すようになっているおかげで、再帰的に評価できる。はじめ実装した時スタックオーバーフローを起こしていたが、それは環境を `dump` するデバッグ関数の仕業だった (このミスのおかげで環境が再帰的になっている様子が理解できた。)

発展 1

動作例

```
# (fun x y z -> x*y*z) 2 3 4;;
- = 24
# fun x y -> x+y;;
- = <fun>
# let f x y = x+y in f 2 3;;
```

```
- = 5
# f 2 3;;
Fatal error: exception Eval.EvalErr
-----
# let f x y = x*y;;
val f = <fun>
```

考察

Parser に (1 つ以上の var) ARROW expr を表す var_arrow_expr などを追加し、
 fun x y z -> e ならば fun x -> fun y -> fun z -> e と解釈されるように変更した。
 let f x y = e, let f x y = e in e' の形も同様の方法でサポートした。

発展 2

動作例

```
# let rec f n = if n = 0 then 0 else n + f(n-1);;
val f = <fun>
# f 10;;
- = 55
# f 20;;
- = 210
```

考察

値の再帰的定義によって定義できるデータを観察すると、無限にループする構造を定義できていることがわかる。ここで問 4 で用いたスライドの方法 2 を思い出すと、この方法で参照を用いて環境をループさせていたことに着目し、この構造を値の再帰的定義を用いて実現できた。評価時の環境を関数 f で拡張した環境と、その f のクロージャ内の関数が一致すればよいので、評価時の環境を env, f で拡張後の環境を env' とすると $env' = (VFun\ f\ n\ env') :: env$ とすればよいことがわかる。

発展 3

動作例

```
# let a = 10;;
val a = 10
# let f = dfun x -> x+a;;
val f = <dfun>
# f 10;;
- = 20
# let a = 20;;
val a = 20
# f 10;;
- = 30
# let fact = dfun n -> if n = 1 then 1 else n*fact (n-1);;
```

```
val fact = <dfun>
# fact 10;;
- = 3628800
# fact 5;;
- = 120
```

考察

動的束縛の関数定義は、クロージャに環境を含めずすべて評価時の環境で評価してやることで実現できる。動的束縛の関数では評価時の環境を用いるので、ARROW 以降の式の部分を評価する際の環境に定義した関数が入っており、自然に再帰関数を定義できる。